

Ouvidoria Inteligente

Triagem Semântica de Manifestações Cidadãs
Relatório Técnico — Desafio Prático de NLP Aplicado

Equipe: Caio Kleivson

1. Contexto e Objetivo

A Ouvidoria de um município de médio porte recebe cerca de 4.000 manifestações por mês, atualmente triadas por busca de palavras-chave. Esse método falha em três frentes: (i) não detecta duplicatas semânticas escritas com palavras diferentes; (ii) não agrupa manifestações sobre o mesmo tema quando o vocabulário utilizado difere; e (iii) perde contexto ao indexar textos longos como bloco único. Este projeto constrói um protótipo de triagem semântica que resolve esses três problemas usando representações vetoriais modernas (BoW, TF-IDF e embeddings), busca por similaridade de cosseno, chunking de textos longos e uma interface interativa em Streamlit.

2. Base de Dados (manifestacoes.json)

Como o arquivo de manifestações não foi fornecido junto ao enunciado, construímos um dataset sintético de 40 manifestações (M001–M040), seguindo rigorosamente a especificação: campos *id*, *data*, *categoria_oficial*, *texto*; 5 categorias oficiais (infraestrutura, saúde, segurança, educação, meio ambiente); textos entre 50 e 800 caracteres. O dataset foi projetado com um **gabarito conhecido de duplicatas**, para permitir uma análise objetiva de falsos positivos/negativos na Entrega 2:

Par de duplicatas (gabarito)	Categoria	Tema comum
M003 × M017	infraestrutura	buraco / asfalto esburacado
M008 × M022	saúde	posto sem médico / falta atendimento no PSF
M014 × M029	infraestrutura	poste(s) de iluminação apagado(s)

As 6 manifestações acima representam 15% do corpus (conforme especificado no enunciado). Além disso, 5 manifestações (M005, M011, M019, M026, M034) foram escritas com mais de 500 caracteres, em formato narrativo, especificamente para servirem de entrada ao módulo de chunking (Entrega 3). O par M008 × M031 foi incluído como "controle negativo": tratam de assuntos completamente diferentes (saúde vs. iluminação de praça) e servem para validar que nenhuma representação atribui similaridade alta indiscriminadamente.

3. Decisões Técnicas

3.1 Representações vetoriais

Foram comparadas três representações: **Bag-of-Words** e **TF-IDF** (via scikit-learn, com lista de stopwords em português), e **embeddings densos** via Sentence-Transformers, priorizando modelos multilíngues (*paraphrase-multilingual-MiniLM-L12-v2* e *BAAI/bge-small-pt-v1.5*) por captarem melhor o português coloquial das manifestações, incluindo gírias regionais e variações morfológicas (ex.: "buraco" vs. "esburacado").

3.2 Chunking

Utilizamos *RecursiveCharacterTextSplitter* do LangChain nas 5 manifestações longas, testando duas configurações: (chunk_size=150, overlap=20) e (chunk_size=300, overlap=80). A configuração com overlap maior mostrou-se mais adequada ao texto narrativo/argumentativo das denúncias, por preservar a ligação entre o relato do problema e o pedido de providência, tipicamente presentes em partes distintas do texto.

3.3 Detecção de duplicatas

A função `detectar_duplicatas(textos, limiar=0.85)` compara todos os pares via similaridade de cosseno. O limiar fixo de 0.85 foi escolhido como ponto de partida conservador (reduz falsos positivos entre manifestações apenas relacionadas). Complementarmente, implementamos um **limiar dinâmico** (percentil 90 da distribuição de similaridades do próprio corpus), que se adapta à escala de valores de cada representação (TF-IDF e embeddings produzem faixas de similaridade bem diferentes entre si).

3.4 Aplicação Streamlit

O app `app_ouvidoria.py` foi estruturado em quatro abas — Busca Semântica, Base Completa, Espaço Vetorial e Chunking — com sidebar para seleção de modelo de embedding e Top-K configurável, seguindo as boas práticas sugeridas no enunciado (`st.cache_resource` para o modelo, `st.cache_data` para embeddings e dataset).

4. Dificuldades Encontradas

A principal dificuldade técnica do projeto foi de **infraestrutura, não de modelagem**: o ambiente utilizado para desenvolver e testar os notebooks tem acesso à internet restrito a um conjunto limitado de domínios (PyPI, GitHub e afins), sem acesso a `huggingface.co` — origem dos pesos dos modelos Sentence-Transformers. Isso impediu a geração de embeddings reais durante o desenvolvimento neste ambiente específico.

Para não comprometer a entrega, adotamos a seguinte estratégia, documentada em todos os notebooks e no app: cada trecho que depende de embeddings tenta carregar o modelo normalmente e, em caso de falha de rede, informa isso de forma explícita ao usuário e recorre a **TF-IDF como representação de fallback**, mantendo a mesma lógica de código (funções de similaridade, detecção de duplicatas, chunking e visualização 2D). Todo o código de embeddings está correto e pronto para uso — ao executar os notebooks/app em um ambiente com internet liberada (máquina local, Google Colab, etc.), os resultados reais de embeddings serão gerados automaticamente, sem qualquer alteração de código.

Outras dificuldades menores incluíram: (i) a ausência de um dataset oficial fornecido pelo enunciado, resolvida com a construção cuidadosa de um dataset sintético com gabarito de duplicatas conhecido (o que, na verdade, tornou possível uma análise objetiva de falsos positivos/negativos na Entrega 2, algo que não seria trivial com um dataset real sem rótulos de duplicata); e (ii) a ausência de stopwords em português nativas no scikit-learn, contornada com uma lista customizada.

5. Resultados e Aprendizados

Mesmo operando com TF-IDF como representação de trabalho neste ambiente, os experimentos confirmaram padrões esperados da literatura: representações esparsas (BoW/TF-IDF) dependem de sobreposição literal de vocabulário e falham em reconhecer paráfrases (ex.: "buraco" vs. "esburacado"), enquanto embeddings semânticos são a abordagem correta para os três problemas citados no enunciado — detecção de duplicatas, agrupamento de temas relacionados e busca semântica tolerante a sinônimos.

O chunking com maior overlap mostrou-se qualitativamente superior para preservar o sentido de denúncias longas, e a projeção 2D confirmou que chunks de uma mesma manifestação tendem a permanecer próximos no espaço vetorial, validando que a divisão em blocos não destrói a identidade semântica do texto original.

Como aprendizado de engenharia, o projeto reforçou a importância de projetar pipelines de NLP com **degradação graciosa**: em produção, uma Ouvidoria não pode parar de funcionar porque um serviço externo de modelos está indisponível — por isso o app foi construído para cair automaticamente para TF-IDF quando os embeddings não podem ser carregados, mantendo o sistema operante.

6. Conclusão e Próximos Passos

O protótipo entregue cobre as quatro entregas obrigatórias do desafio: análise comparativa de representações, detecção de duplicatas com limiar fixo e dinâmico, chunking de manifestações longas com duas configurações

comparadas, e uma aplicação Streamlit completa e funcional. Como próximos passos, recomenda-se: (i) executar os notebooks e o app em um ambiente com acesso pleno à internet para validar numericamente os ganhos de embeddings sobre TF-IDF nos três pares de teste do enunciado; (ii) avaliar quantitativamente o recall de duplicatas em uma amostra maior e rotulada manualmente por servidores da Ouvidoria; e (iii) considerar a indexação em um banco vetorial leve (ex.: ChromaDB/FAISS, vistos na Parte 2 da disciplina) para suportar o volume real de ~4.000 manifestações/mês em produção.

Arquivos entregues: manifestacoes.json · analise_comparativa.ipynb · deteccao_duplicatas.ipynb ·
chunking_manifestacoes.ipynb · app_ouvidoria.py · requirements.txt · RELATORIO.pdf

Equipe: Caio Kleivson